

Exercise 1.i

Finding the equilibrium analitically:

```
syms c(t) V Q k F;  
ode(t) = diff(c(t),t) == (F/V)-(Q*c/V)-(k*c^2);  
F=200;V=10;Q=1;k=0.1; % to subs in symbolic function  
ode(t)=subs(ode(t))
```

ode(t) =

$$\frac{\partial}{\partial t} c(t) = -\frac{c(t)^2}{10} - \frac{c(t)}{10} + 20$$

If we let $c(t) = x$, then we can solve for $\frac{dc}{dt} = -\frac{c^2}{10} - \frac{c}{10} + 20 = -\frac{x^2}{10} - \frac{x}{10} + 20 = 0$:

```
syms g(x);  
g(x) = -x^2/10-x/10+20
```

g(x) =

$$-\frac{x^2}{10} - \frac{x}{10} + 20$$

```
solve(g,x);  
conc_equilib = (3*sqrt(89)/2)-0.5
```

conc_equilib = 13.6510

```
cond = c(0) == 0; % initial concentration  
conc(t)=dsolve(ode(t),cond) % analytical solution
```

conc(t) =

$$\frac{3 \sqrt{89} \tanh\left(\frac{\sqrt{89} \left(267 t + 20 \sqrt{89} \operatorname{atanh}\left(\frac{\sqrt{89}}{267}\right)\right)}{1780}\right)}{2} - \frac{1}{2}$$

```
f=@func;  
[a,b]=ode45(f,[0:0.5:15],0);  
numerical = runge_kutta(f,[0:0.1:15],0);
```

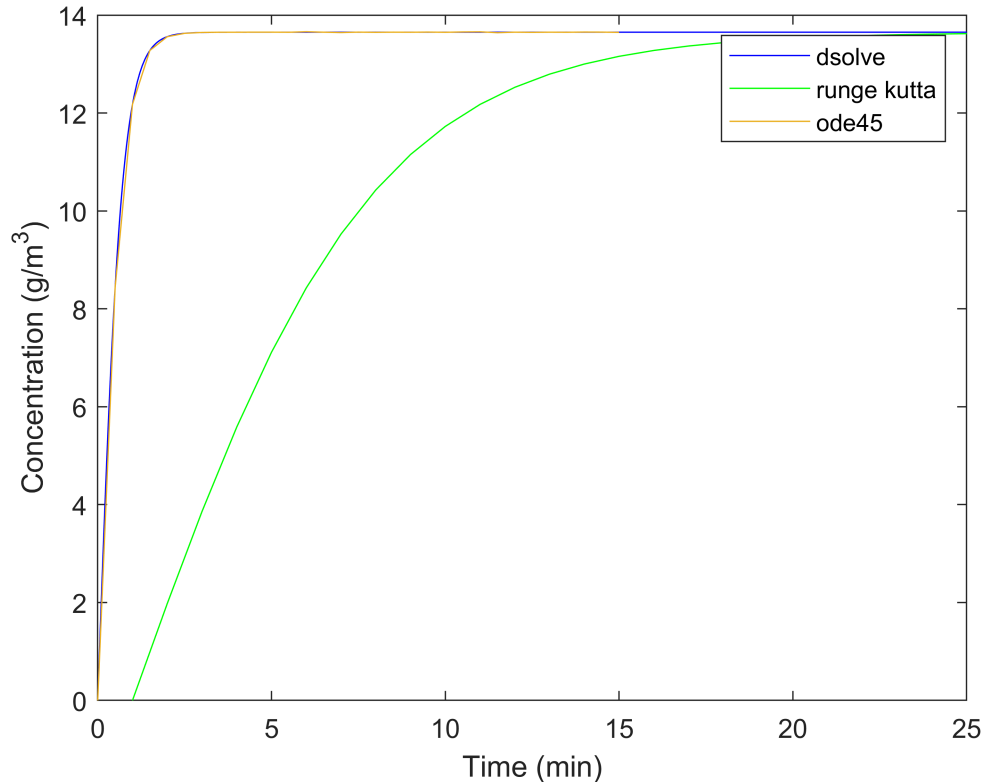
```
fplot(conc,[0 25], 'b')  
xlim([0 25]);  
ylim([0 14]);  
hold on
```

```
plot(numerical, 'g')  
xlim([0 25]);  
ylim([0 14]);  
hold on  
plot(a,b)  
xlim([0 25]);  
ylim([0 14]);
```

```

legend("dsolve","runge kutta","ode45");
xlabel('Time (min)');
ylabel('Concentration (g/m^3)');

```



The steady state concentration is $13.6510 \frac{g}{m^3}$. By the plot above, note that the analytical solution grows and approach the steady state concentration more rapidly than the Runge Kutta algorithm. However, the Runge Kutta algorithm does approach the steady state later in time.

```

function [y] = runge_kutta(f,x,y0)
n=length(x);
m=length(y0);
y=zeros(m,n);
y(:,1)=y0; % the first column is equal to the initial value vector
for i=1:n-1 % for i from 1 to length of IV - 1
    h = x(i+1)-x(i); % cal
    xi=x(i);
    yi=y(:,i);
    k1 = f(xi,yi);
    k2 = f(xi+0.5*h,yi+0.5*k1*h);
    k3 = f(xi+0.5*h,yi+0.5*k2*h);
    k4 = f(xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6; % this is a matrix
end
end

function [dcdt] = func(t,c)

```

```
V=10;F=200;Q=1;k=0.1;  
dcdt=zeros(1,1);  
dcdt(1) = (F/V)-(Q*c./V)-k.*c.^2;  
end
```

Exercise 1 part (ii):

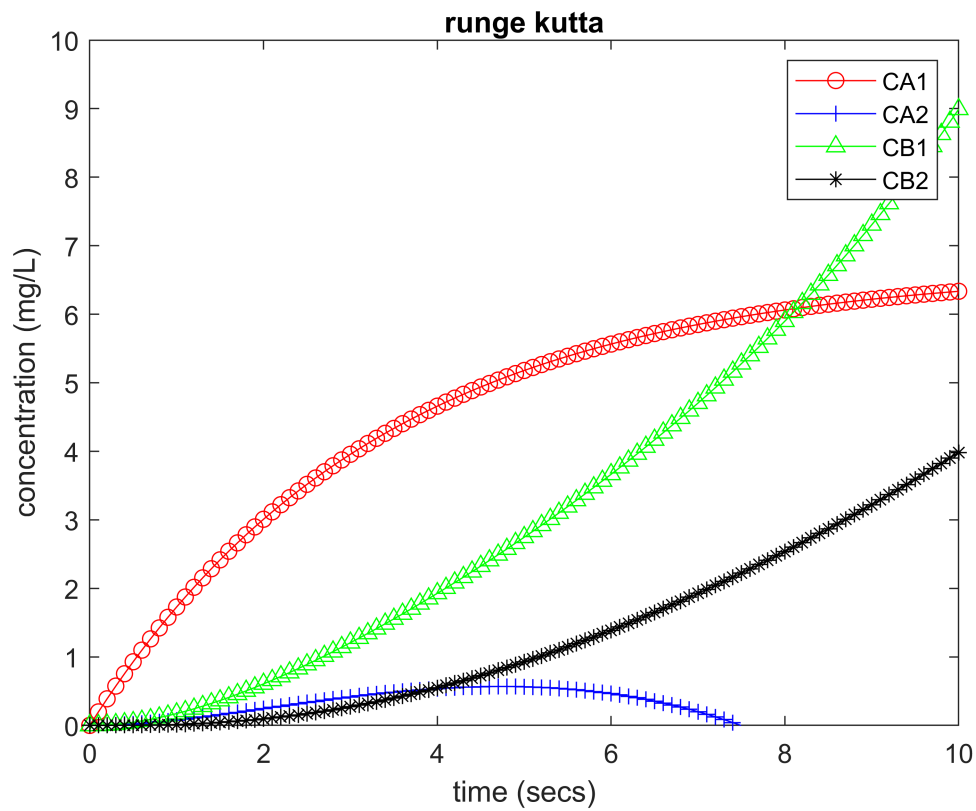
Physical meaning of each equation:

1. The change in concentration of A (mg/L) at the outlet of reactor 1 with respect to time (minutes) that enters reactor 2 will be given by the amount of concentration of A that will be in the reactor during the residence time $(\frac{CA_0 - CA_1}{\tau})$ minus the amount of A that is reacting into B as time passes (kCA_1).
2. The change in concentration of B (mg/L) with respect to time (minutes), at the outlet of reactor 1 and going into reactor 2 is given by the amount of A that reacted to B that is in the outlet of reactor 1 (kCA_1) minus the amount of B that is inside the reactor $(\frac{CB_1}{\tau})$.
3. The change in concentration of A (mg/L) with respect to time (minutes) at the outlet of reactor 2 is given by the amount of time A is inside the reactor 2 $(\frac{CA_1 - CA_2}{\tau})$ minus the amount of A in the outlet of Reactor 2 that is reacting into B as time passes (kCA_2).
4. The change in concentration of B (mg/L) at the outlet of reactor 2 with respect to time (minutes) is given by the the amount of A at the outlet of reactor 2 that is reacting into B as time passes (kCA_2) and the amount of B that is inside reactor 2 $(\frac{CB_1 - CB_2}{\tau})$.

```
c0 = [0; 0; 0; 0]; % initial condition
t = [0:0.1:10];
[c] = runge_kutta(@chemical_reactor,t,c0);
c1=c(1,:);
c2=c(2,:);
c3=c(3,:);
c4=c(4,:);

% runge kutta:
figure(1)
plot(t,c1,'-ro',t,c2,'-b+',t,c3,'-g^',t,c4,'-k*')
legend('CA1','CA2','CB1','CB2');
xlabel('time (secs)');
ylabel('concentration (mg/L)');
title('runge kutta')
hold off;
ylim([0,10])

% ode 45:
figure(2)
ode45(@chemical_reactor,[0 10],c0)
```

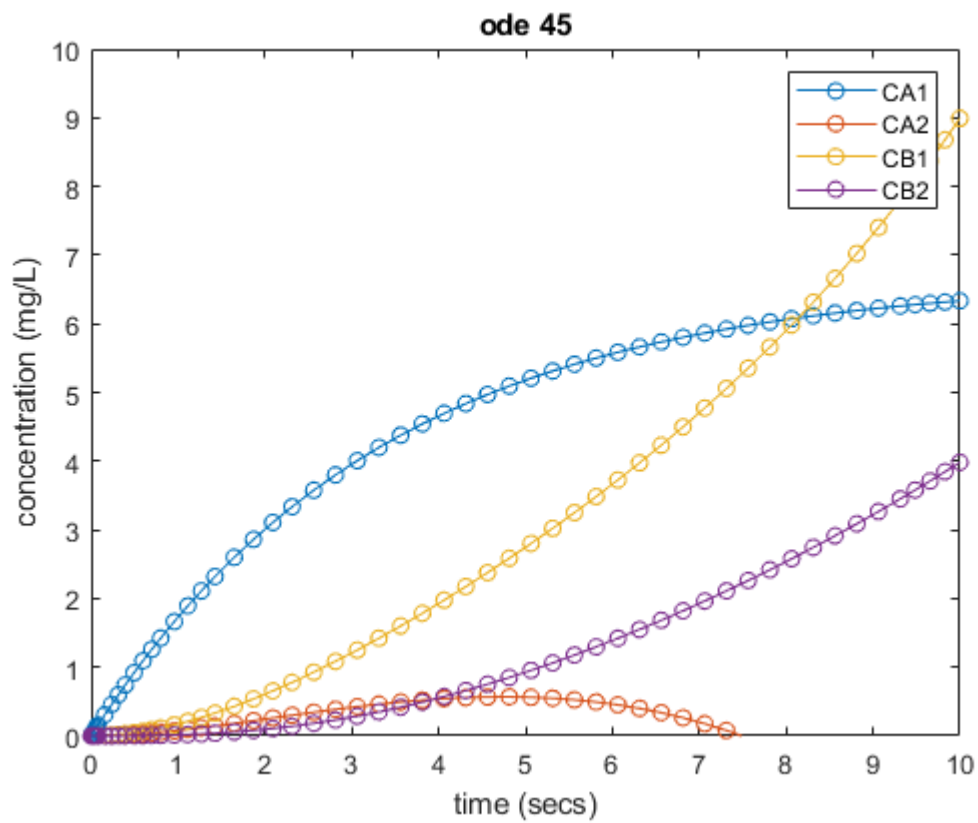


```

legend('CA1','CA2','CB1','CB2');
title('ode 45');
xlabel('time (secs)');
ylabel('concentration (mg/L)');
ylim([0,10]);

% both
figure(3)
plot(t,c1,'-ro',t,c2,'-b+',t,c3,'-g^',t,c4,'-k*');
xlabel('time (secs)');
ylabel('concentration (mg/L)');
hold on
ode45(@chemical_reactor,[0 10],c0)

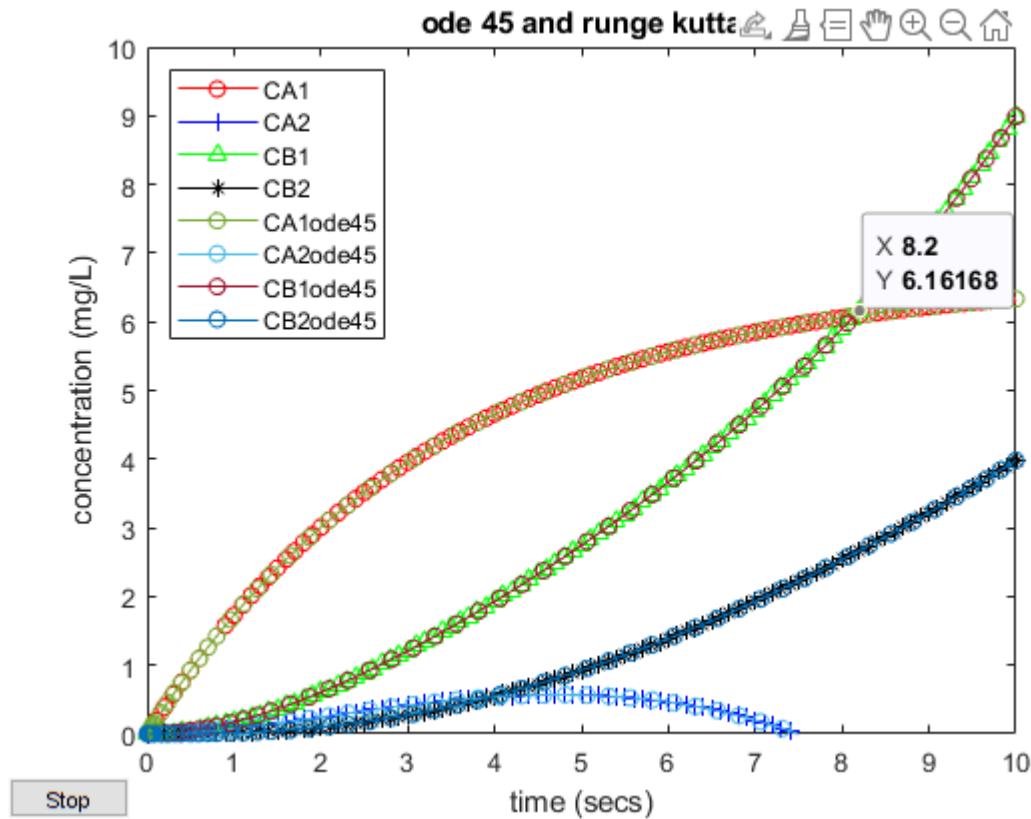
```



```

title('ode 45 and runge kutta');
ylim([0,10])
legend('CA1','CA2','CB1','CB2','CA1ode45','CA2ode45','CB1ode45','CB2ode45','Location','northwest')

```



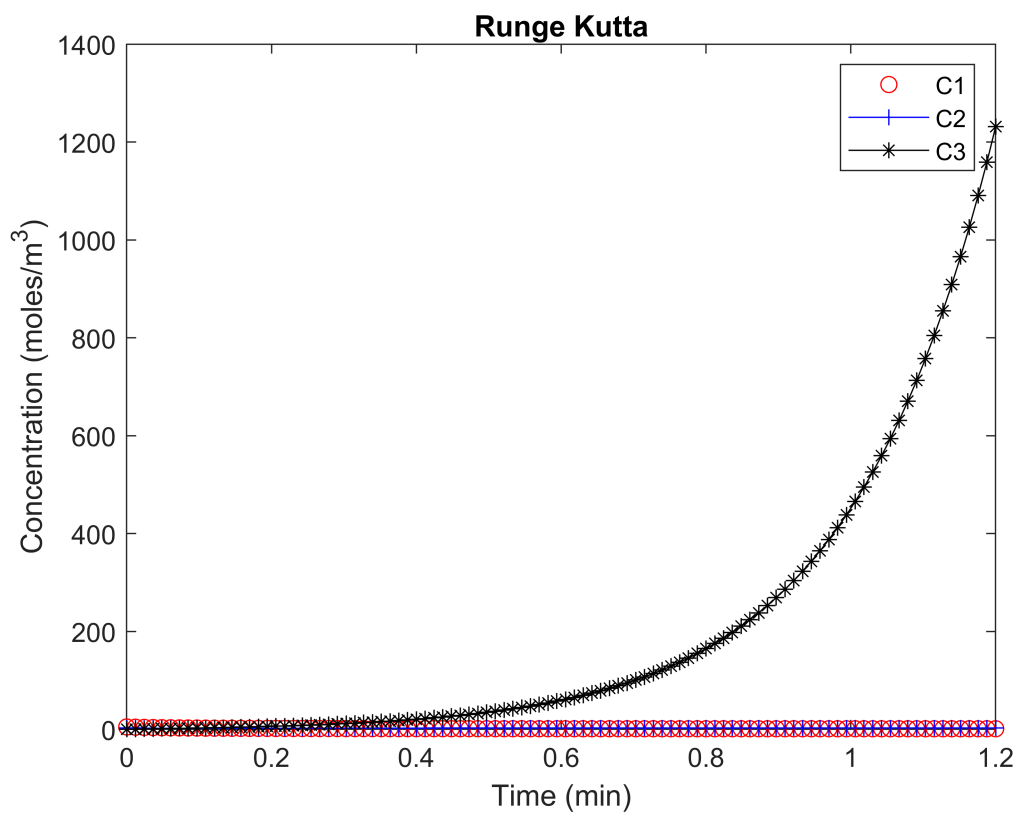
```
function [dcdt] = chemical_reactor(t,c)
dcdt = zeros(4,1); % 4 equations
dcdt(1)=(1/5)*(10-c(1))-0.1*c(1);
dcdt(2)=-(1/5)*c(3)+0.1*c(1);
dcdt(3)=(1/5)*(c(1)-c(2))-0.1*c(2);
dcdt(4)=(1/5)*(c(3)-c(4))+0.1*c(2);

end

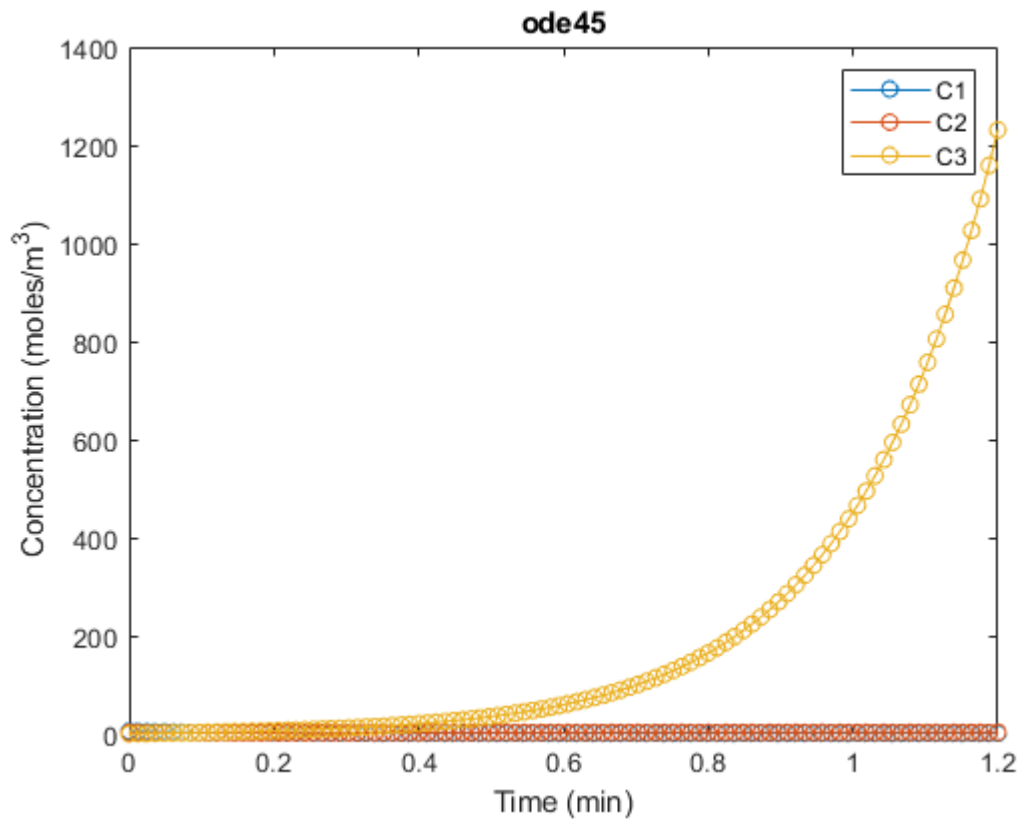
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = df(xi,yi);
    k2 = df(xi+0.5*h,yi+0.5*k1*h);
    k3 = df(xi+0.5*h,yi+0.5*k2*h);
    k4 = df(xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
end
```


Exercise 2

```
t=linspace(0,1.2);  
C=[5; 3; 0];  
g=@odesyst2;  
[result]=runge_kutta(g,t,C);  
C1 = result(1,:);  
C2 = result(2,:);  
C3 = result(3,:);  
plot(t,C1,'ro',t,C2,'-b+',t,C3,'-k*')  
title('Runge Kutta')  
xlabel('Time (min)')  
ylabel('Concentration (moles/m^3)')  
legend('C1','C2','C3');
```



```
ode45(g,t,C)  
title('ode45')  
xlabel('Time (min)')  
ylabel('Concentration (moles/m^3)')  
legend('C1','C2','C3');
```



ode45 and runge_kutta give the same result.

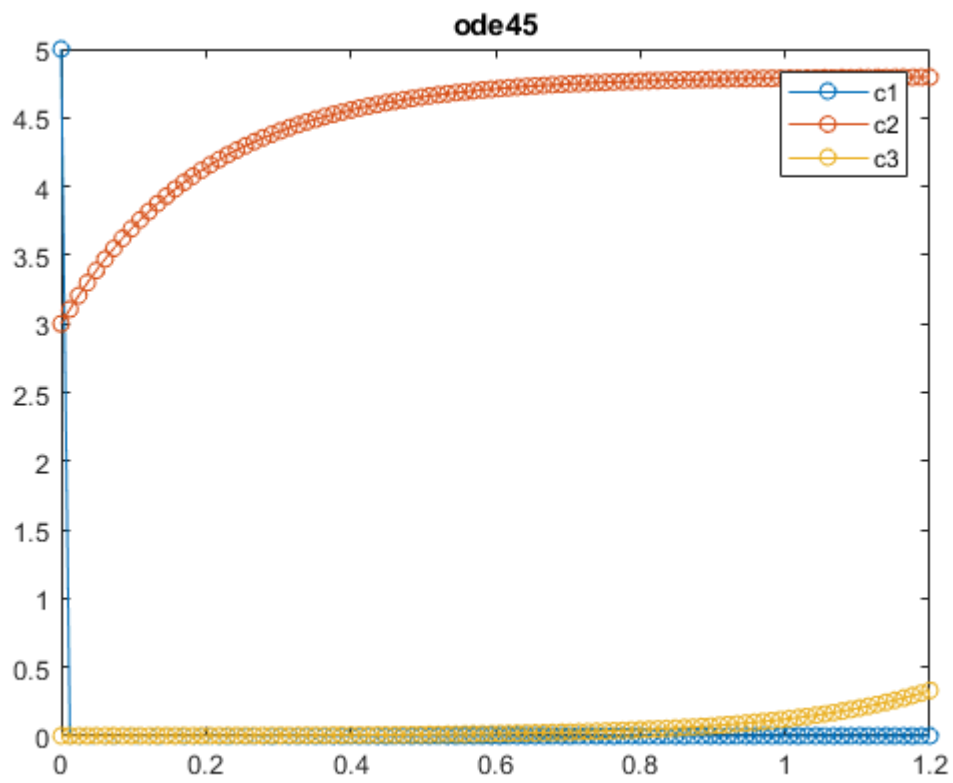
Solving the ode system using ode45 and ode23s with $\alpha_1 = 100000$, $\alpha_2 = 1$, $\alpha_3 = 10$.

```
[c]=runge_kutta(@odesyst,t,C)
```

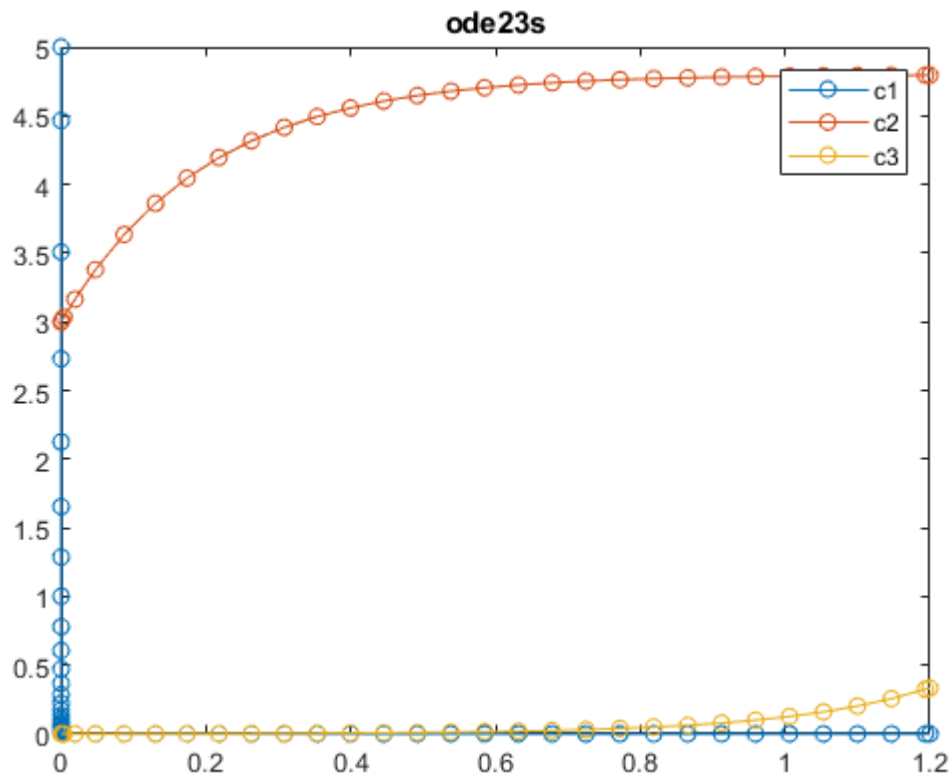
```
c = 3×100
1022 ×
    0.0000    -2.1851    -Inf      NaN      NaN      NaN      NaN      NaN ...
    0.0000    -0.0000    -Inf      NaN      NaN      NaN      NaN      NaN
    0         0.0002     Inf       NaN      NaN      NaN      NaN      NaN
```

Using Rungw Kutta does not work for n=4. For n=100 it will also not work because we have a stiff ode system.

```
t=linspace(0,1.2);
C=[5; 3; 0];
ode45(@odesyst,t,C)
title('ode45')
legend('c1','c2','c3')
```



```
ode23s(@odesyst,[0 1.2],C)
title('ode23s')
legend('c1','c2','c3')
```



It took 6 seconds for ode45 to run. In contrast, ode23s took 0.41 seconds.

```
function [dcdt] = odesyst(t,c)
c01=3.2;c02=4.8;Q=10; V=2;
dcdt=zeros(3,1);
dcdt(1)=(Q/V)*(c01-c(1))-100000*c(1)*c(2);
dcdt(2)=(Q/V)*(c02-c(2))-1*c(1)*c(2);
dcdt(3)=(Q/V)*c(3)+10*c(1)*c(2);
end
function [dcdt] = odesyst2(t,c)
c01=3.2;c02=4.8;Q=10; V=2;
dcdt=zeros(3,1);
dcdt(1)=(Q/V)*(c01-c(1))-2.5*c(1)*c(2);
dcdt(2)=(Q/V)*(c02-c(2))-2.5*c(1)*c(2);
dcdt(3)=(Q/V)*c(3)+2.5*c(1)*c(2);
end
```

```
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
```

```

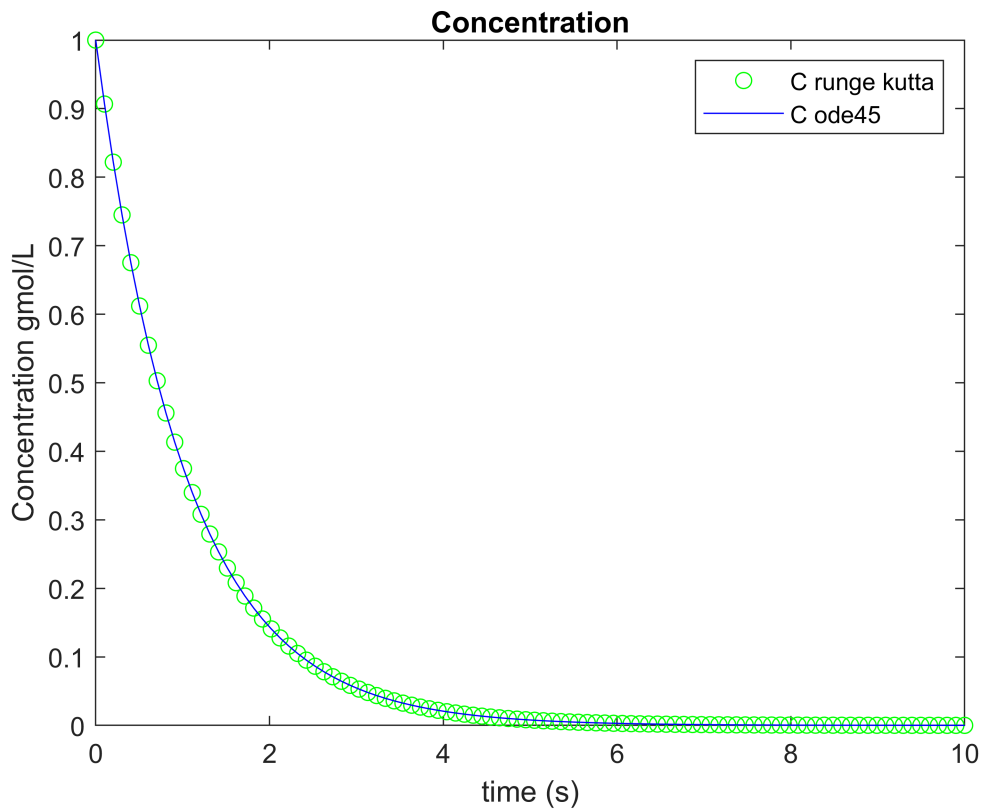
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
end

```

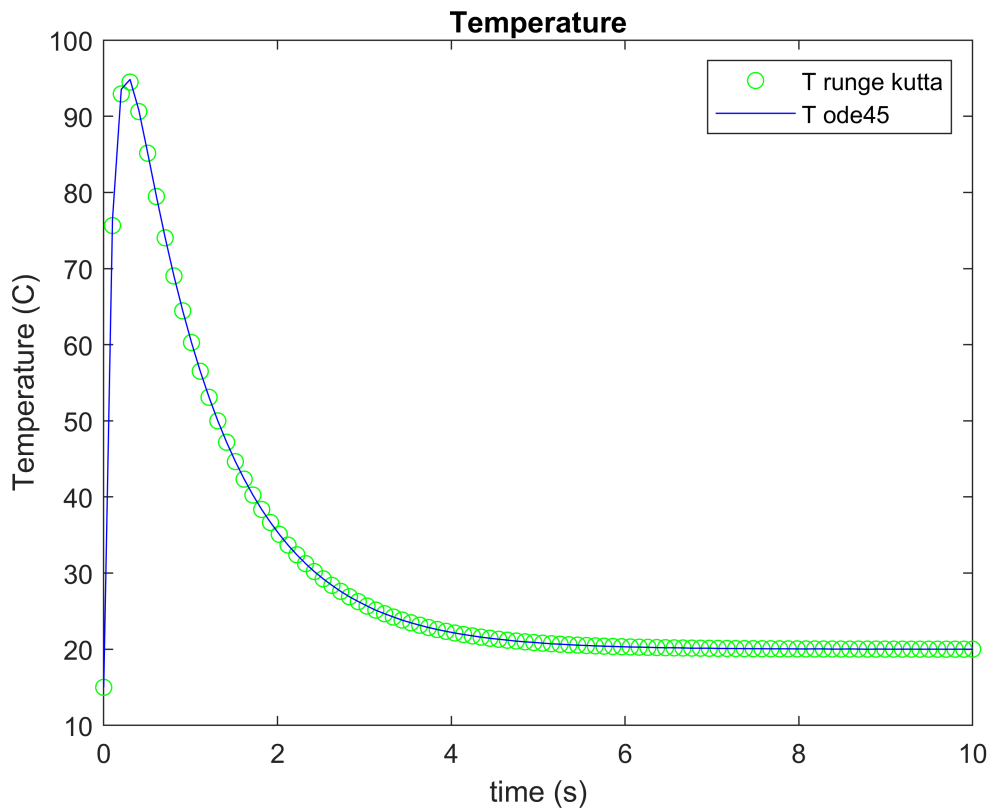
Exercise 3

```
initialcond=[1; 15];% initial conditions
t=linspace(0,10);
func=@odesystem;
[result] = runge_kutta(func,t,initialcond);
C=result(1,:);
T=result(2,:);
[a,b]=ode45(func,t,initialcond);
C_ode45=b(:,1);
T_ode45=b(:,2);

plot(t,C,'go',t,C_ode45,'-b')
title('Concentration');
legend('C runge kutta','C ode45')
xlabel('time (s)');
ylabel('Concentration gmol/L');
```



```
plot(t,T,'go',t,T_ode45,'-b')
title('Temperature');
legend('T runge kutta','T ode45')
xlabel('time (s)');
ylabel('Temperature (C)');
```



```
function [dpdt] = odesystem(t,p)
dpdt=zeros(2,1);
C=p(1); T=p(2);
dpdt(1)= -exp(-10/(T+273))*C; % conc ode
dpdt(2)=1000*exp(-10/(T+273))*C-10*(T-20); % temp ode
end
```

```
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
```

```
y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;  
end  
end
```

Exercise 4:

Reducing ODE:

$$y(1) = x$$

$$\frac{d(y(1))}{dt} = \frac{dx}{dt}$$

$$y(2) = \frac{d(y(1))}{dt}$$

$$\frac{d(y(2))}{dt} = \frac{d^2(y(1))}{dt^2}$$

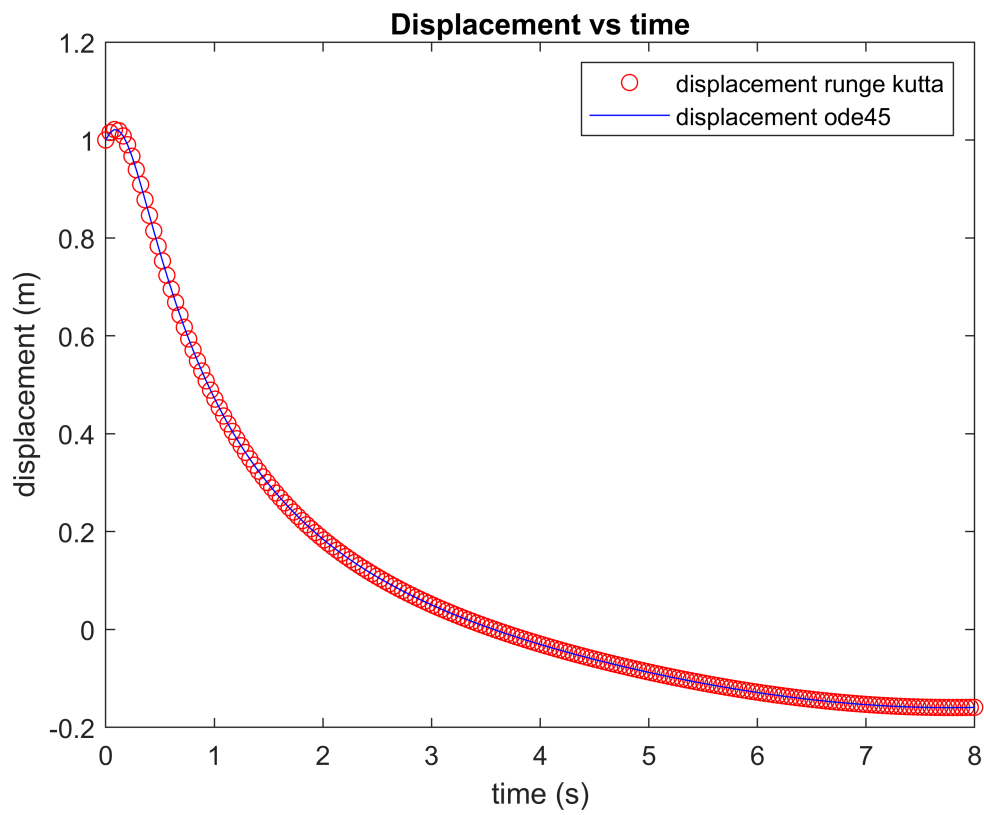
Then our system becomes:

$$m * \frac{dy(2)}{dt} + a * \text{sign}(y(2)) * (y(2))^2 + b * y(1)^3 = 0$$

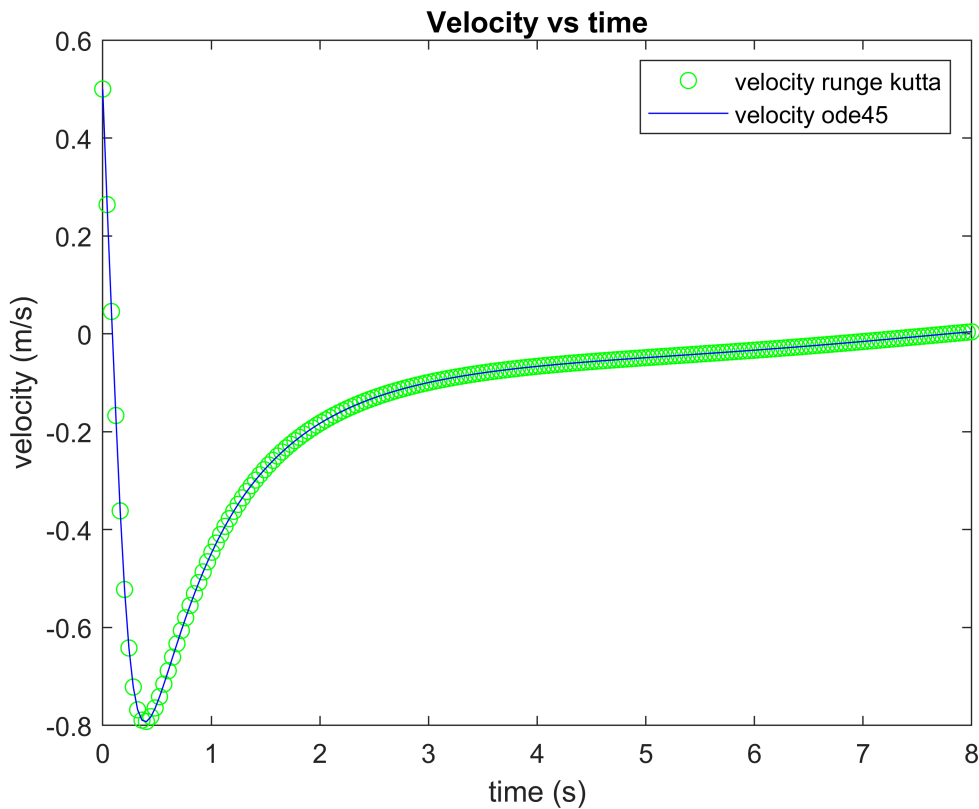
```
initialcond=[1; 0.5];
t=linspace(0,8,200);
func=@disp;
[result]=runge_kutta(func,t,initialcond);
displacement = result(1,:);
velocity = result(2,:);

[a b]=ode45(func,t,initialcond);
displacement_ode45=b(:,1);
velocity_ode45=b(:,2);

plot(t,displacement,'ro',t,displacement_ode45,'-b')
title('Displacement vs time');
legend('displacement runge kutta','displacement ode45');
xlabel('time (s)');
ylabel('displacement (m)');
```



```
plot(t,velocity,'go',t,velocity_ode45,'-b')
title('Velocity vs time');
legend('velocity runge kutta','velocity ode45');
xlabel('time (s)');
ylabel('velocity (m/s)');
```



```
function [dydt]=disp(t,y)
dydt=zeros(2,1);
m=1;a=5;b=5;
dydt(1)=y(2);
dydt(2)=(-1/m)*(a*sign(y(2))*y(2)^2+b*y(1)^3);
end
```

```
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
```

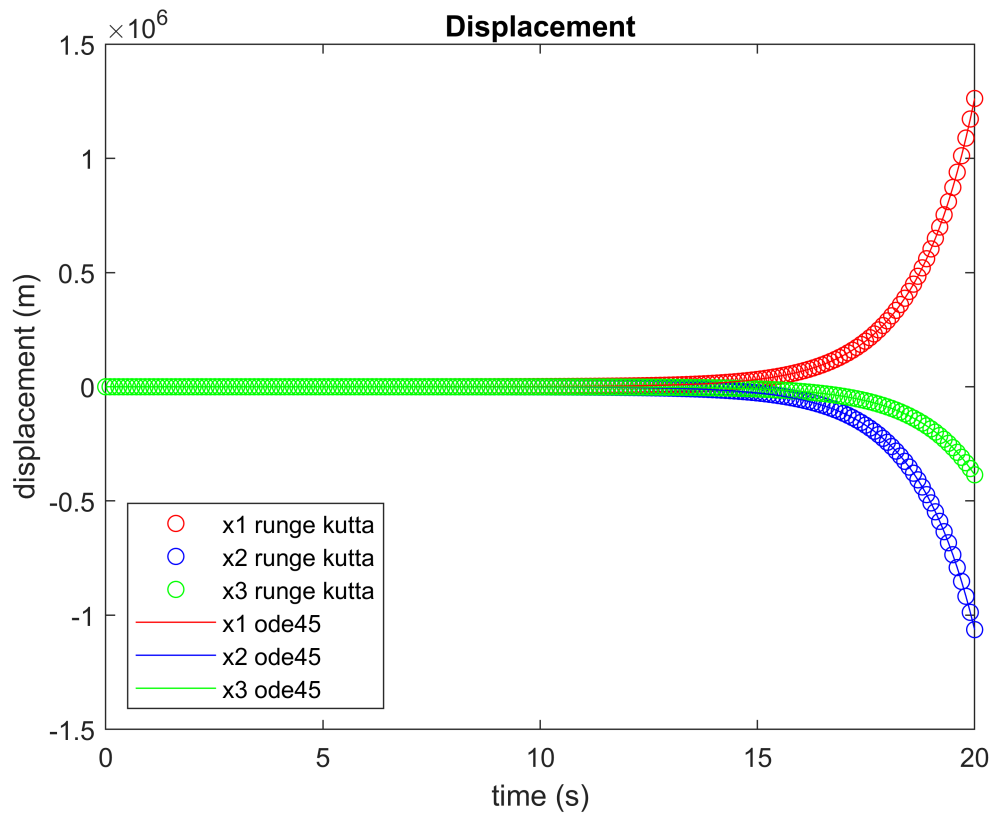
end

Exercise 5

```
initialcond = [0; 0; 0; 1; 0; 0]; % initial conditions
t=linspace(0,20,200);
func = @odesyst;
[a b]=ode45(func,t,initialcond);
ode45x1 = b(:,1);
ode45x2 = b(:,2);
ode45x3 = b(:,3);
ode45velx1 = b(:,4);
ode45velx2 = b(:,5);
ode45velx3 = b(:,6);

[result]=runge_kutta(func,t,initialcond);
x1 = result(1,:);
x2 = result(2,:);
x3 = result(3,:);
velx1 = result(4,:);
velx2 = result(5,:);
velx3 = result(6,:);

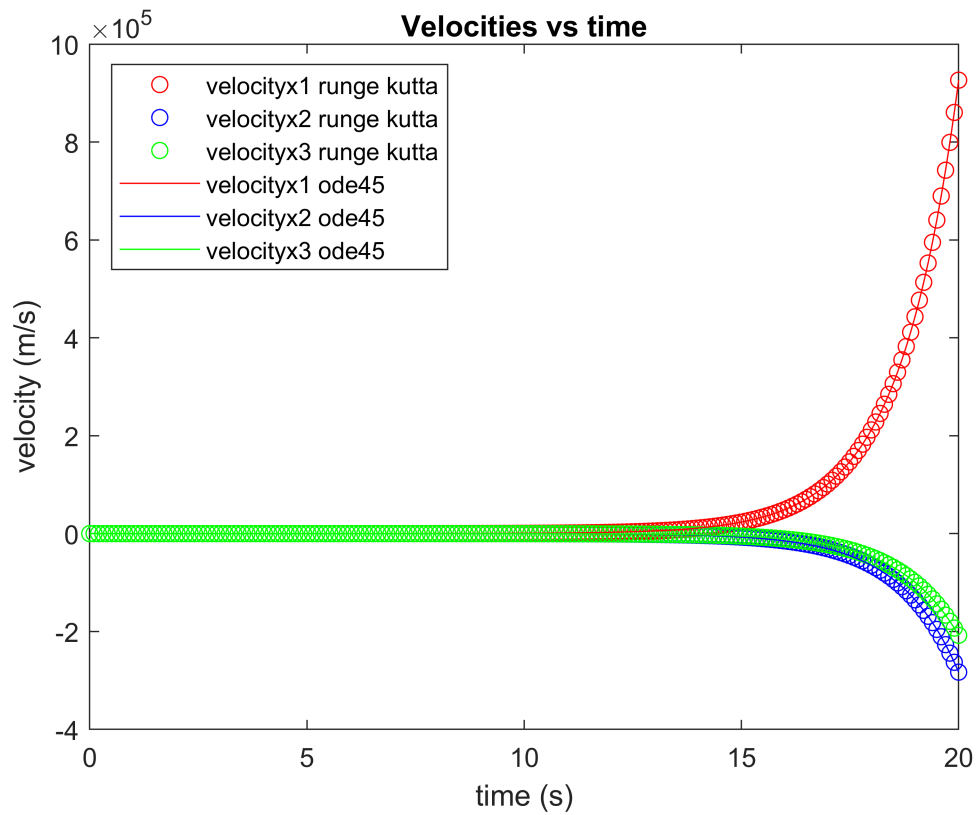
plot(t,x1,'ro',t,x2,'bo',t,x3,'go',t,ode45x1,'r',t,ode45x2,'b',...
      t,ode45x3,'g')
title('Displacement');
xlabel('time (s)');
ylabel('displacement (m)');
legend('x1 runge kutta','x2 runge kutta','x3 runge kutta',...
       'x1 ode45','x2 ode45','x3 ode45','Location','southwest')
```



```

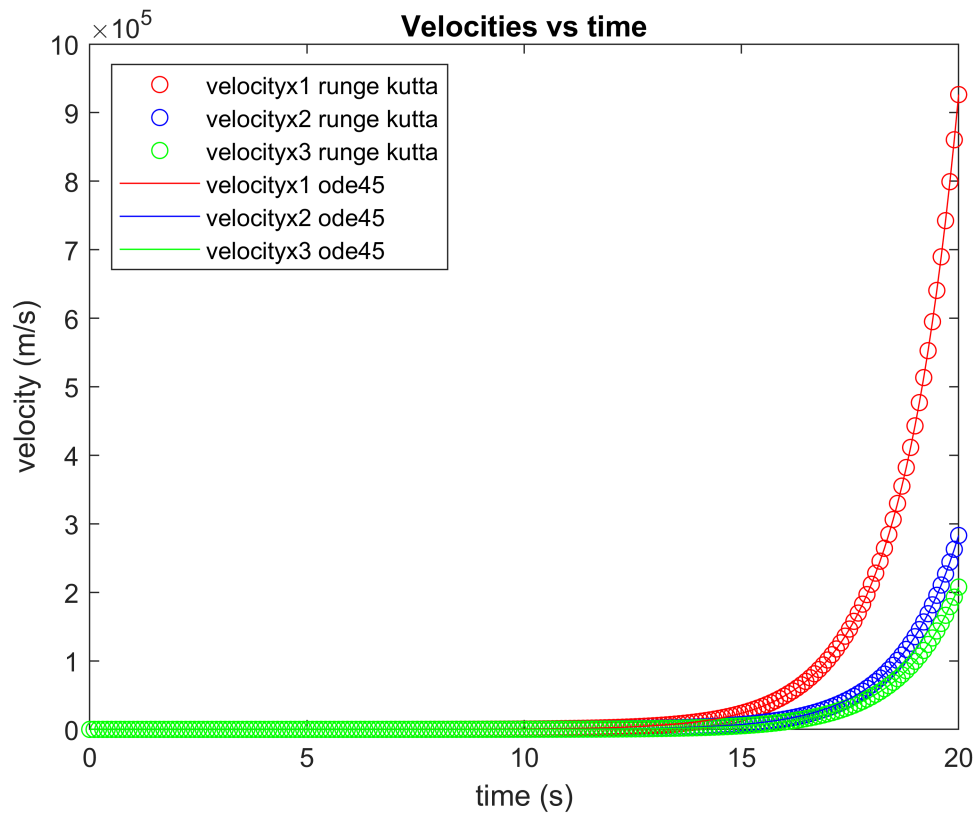
plot(t,velx1,'ro',t,velx2,'bo',t,velx3,'go',t,ode45velx1,'r',t,ode45velx2,'b',...
     t,ode45velx3,'g')
title('Velocities vs time');
xlabel('time (s)');
ylabel('velocity (m/s)');
legend('velocityx1 runge kutta','velocityx2 runge kutta','velocityx3 runge kutta',...
      'velocityx1 ode45','velocityx2 ode45','velocityx3 ode45','Location','northwest')

```



We can also make a graph of the absolute value of the velocities, since we already know the direction of the displacements by the first graph:

```
plot(t,abs(velx1),'ro',t,abs(velx2),'bo',t,abs(velx3),'go',t,abs(ode45velx1),'r',t,abs(ode45velx2),'b',t,abs(ode45velx3),'g');
title('Velocities vs time');
xlabel('time (s)');
ylabel('velocity (m/s)');
legend('velocityx1 runge kutta','velocityx2 runge kutta','velocityx3 runge kutta',...
       'velocityx1 ode45','velocityx2 ode45','velocityx3 ode45','Location','northwest')
```



```
function [dydt] = odesyst(t,y)

k1=3000; k2=2400; k3=1800;
m1=12000; m2=10000; m3=8000;

x1=y(1);x2=y(2);x3=y(3);
dydt=zeros(6,1);

dydt(1) = y(4); %dx1/dt
dydt(2) = -(k1/m1)*x1+(k2/m1)*(x2-x1); %d^2x1/dt^2

dydt(3) = y(5); % dx2/dt
dydt(4) = (k2/m2)*(x1-x2)+(k3/m2)*(x3-x2); %d^2x2/dt^2

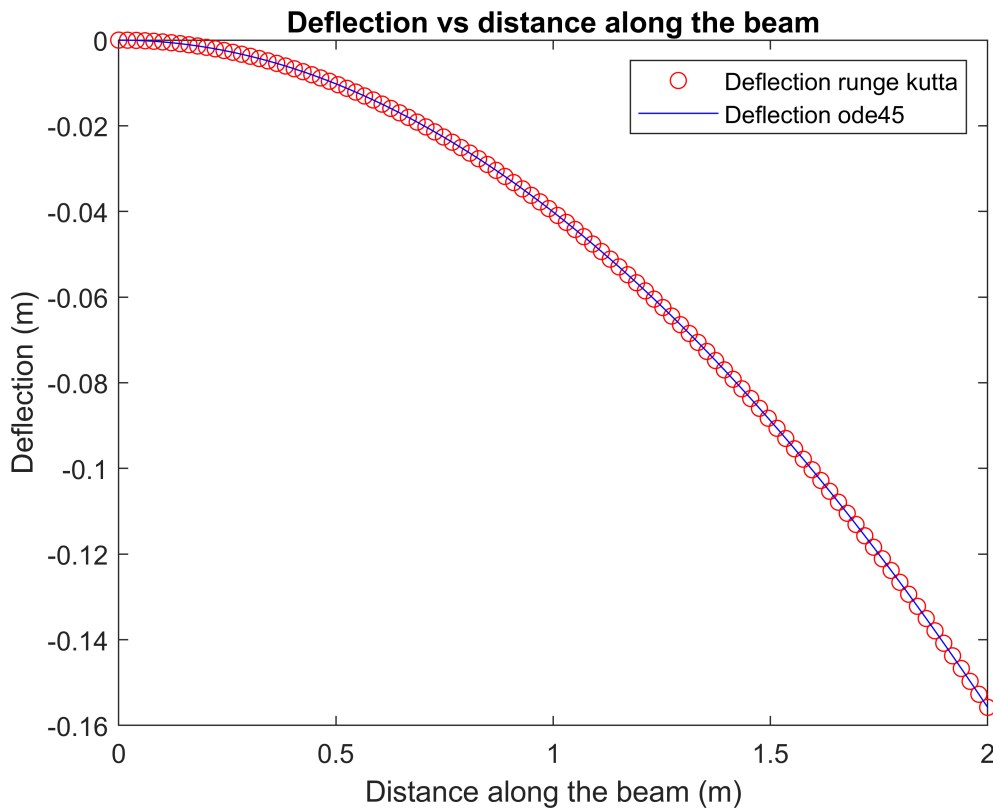
dydt(5) = y(6); % dx3/dt
dydt(6) = (k3/m3)*(x2-x3); % d^2x3/dt^2
end

function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
```

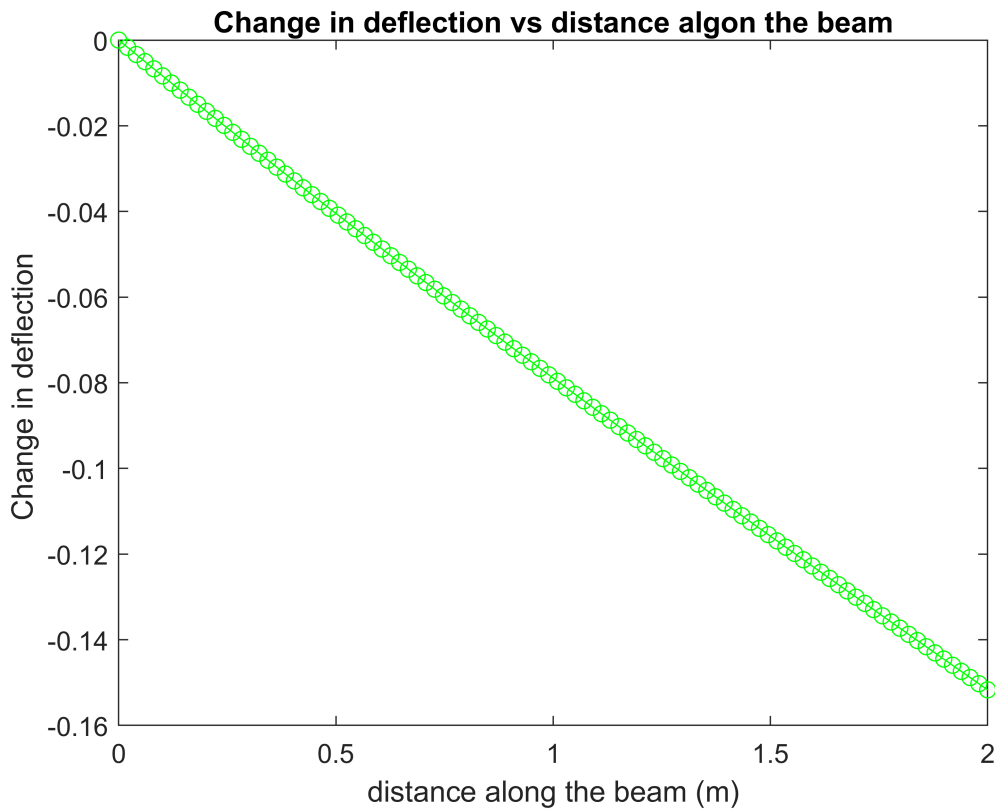
```
h = x(i+1)-x(i);
xi=x(i);
yi=y(:,i);
k1 = feval(df,xi,yi);
k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
k4 = feval(df,xi+h,yi+k3*h);
y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
end
```

Exercise 6:

```
bc = [0; 0];  
t=linspace(0,2);  
func=@odesyst;  
[a,b] = ode45(func,t,bc);  
  
yode45 = b(:,1);  
dydxode45 = b(:,2);  
  
[results]=runge_kutta(func,t,bc);  
y = results(1,:);  
dydx = results(2,:);  
  
plot(t,y,'ro',t,yode45,'b')  
title('Deflection vs distance along the beam')  
legend('Deflection runge kutta','Deflection ode45')  
xlabel('Distance along the beam (m)')  
ylabel('Deflection (m)')
```



```
plot(t,dydx,'go',t,dydxode45,'g')  
title('Change in deflection vs distance along the beam')  
xlabel('distance along the beam (m)')  
ylabel('Change in deflection ')
```



```
function [dydx] = odesyst(t,y)
L=2; rhog=100; EI=2400;

dydx = zeros(2,1);

dydx(1) = y(2); % dydx

dydx(2) = (1/EI)*(-rhog*sqrt( (1+(y(2)))^3)*(y(1)*(y(1)-L/2)+(L^2/2)));%d^2y/dx^2

end

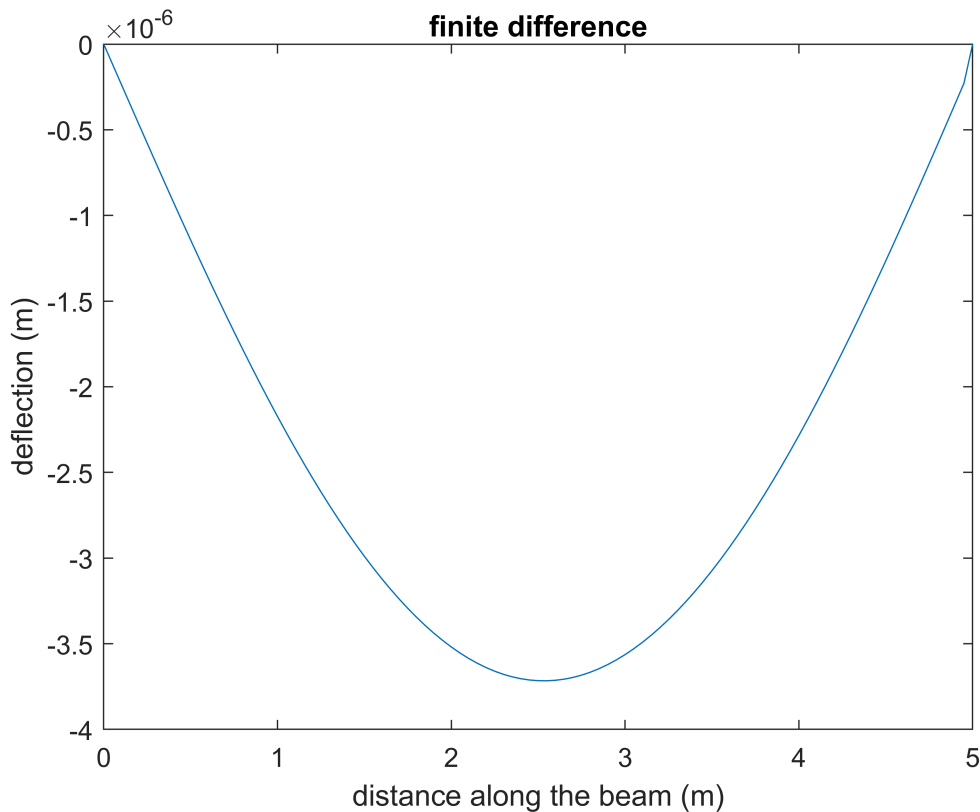
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
```

```
k4 = feval(df,xi+h,yi+k3*h);  
y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;  
end  
end
```

Exercise 7

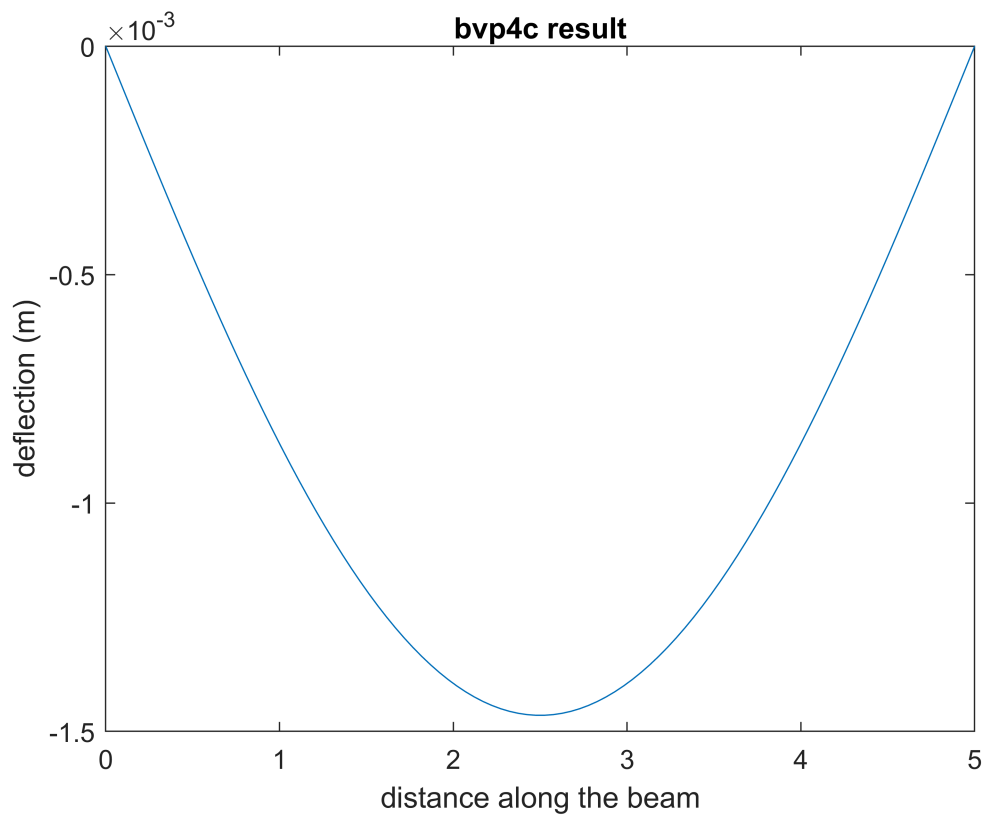
Part (i):

```
x = [0:0.05:5];  
length(x);  
h=1e-3;  
n=2;  
[result]=fda_heat(x);  
  
plot(x,result)  
title('finite difference');  
xlabel('distance along the beam (m)')  
ylabel('deflection (m)');
```



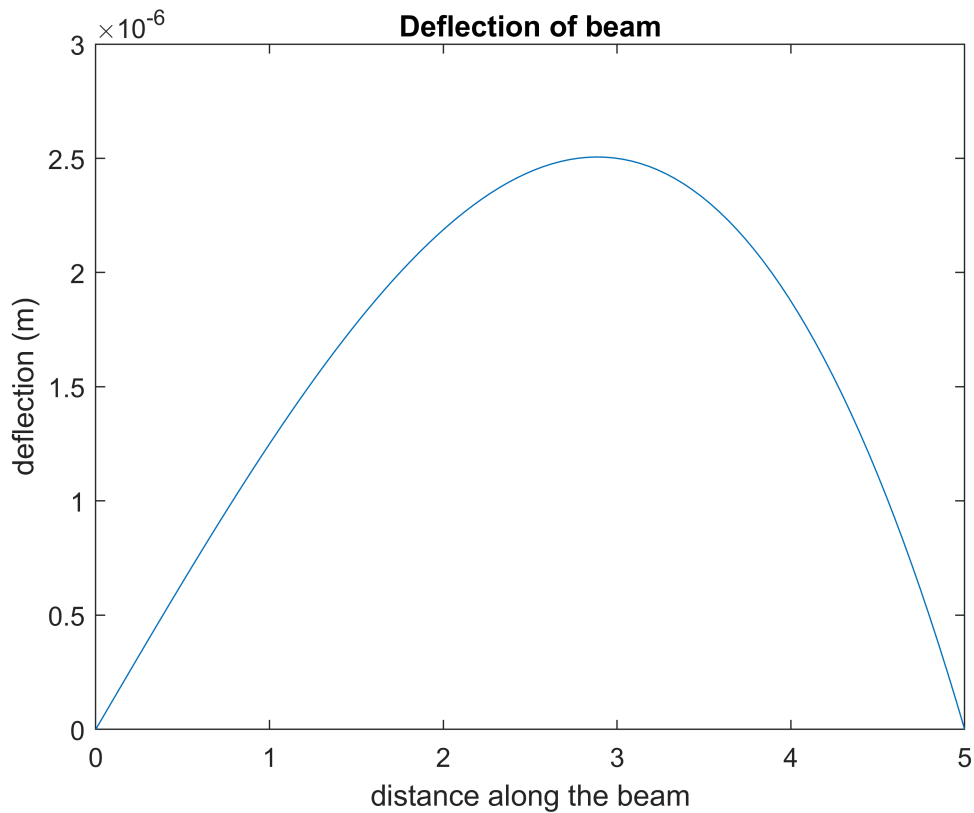
Part (ii):

```
xinit = linspace(0,5);  
yinit=[0 100]; %  
% matlab solution  
solinit = bvpinit(xinit,yinit);  
[result2]=bvp4c(@deflec, @eqbc, solinit);  
plot(result2.x,result2.y(1,:))  
title('bvp4c result');  
xlabel('distance along the beam');  
ylabel('deflection (m)');
```



part (iii):

```
xinit = linspace(0,5);
yinit=[0 100]; %
% matlab solution
solinit = bvpinit(xinit,yinit);
[result3]=bvp4c(@partii, @eqbc, solinit);
plot(result3.x,result3.y(1,:))
title('Deflection of beam')
xlabel('distance along the beam');
ylabel('deflection (m)');
```



```
function [dydx]=partii(x,z)
L=5;q=1500;E=20*10^9;
h=10*10^(-2);w=5*10^(-2);
I=h*w^3/12;
dydx=zeros(2,1);
dydx(1) = z(2);
dydx(2) = (1/E*I)*(-q/2)*sqrt((1+z(2)^2)^3)*(x-L*x);
end

function [res] = eqbc(ya,yb) % initial condition
res = [ya(1); yb(1)];
end
```

```
function [g] = guess(x)
g = [sin(x); cos(x)];
end

function [dydx] = deflec(x,z)
L=5;q=1500;E=20*10^9;I=(5*10^(-2))*(10*10^(-2))/12;
const=-q/(2*E*I);
dydx=zeros(2,1);
dydx(1) = z(2);
dydx(2) = const*(x^2-L*x);
end
```

```

function [T] = fda_heat(x)
y0=0;yL=0;
L=5; q=1500; E=20*10^9; I=(5*10^(-2))*(10*10^(-2))/12;
const=-q/(2*E*I); dx=x(2)-x(1);

m=length(x);
n=m-2; % only solving for the interior points
ld=ones(n,1); %lower diagonal is = 1
ud=ones(n,1);% upped diagonal is 1

d=zeros(n,1); % to initialize the vector,dagonal
d(1:n)=-2;

b=zeros(n,1);% initializing
b(1)=dx^2*(const*(x(1)^2-L*x(1)))-y0; % output vector, y0 bc

b(n)= dx^2*const*(x(n)^2-L*x(n))-yL; %BC
for i=1:n-1
    b(i)=dx^2*const*dx^2*(x(i)^2-L*x(i));
end

%[ld d ud] marix nby3, n = size of matrix
A=my_spdiags([ld d ud],[-1,0,1]); % built in function that take the diagonals of matrix, use my
y=A\b;
T(2:m-1)=y;
T(1)=y0;
T(m)=yL;
end

function [A] = my_spdiags(A,id)
[n m] = size(A);
B = zeros(n,m);
for i=1:m
    if (id(i) <= 0)
        B(1:n+id(i),i) = A(-id(i)+1:n,i);
    else
        B(1+id(i):n,i) = A(1:n-id(i),i);
    end
end
% construct matrix
A = spdiags(B,id,n,n);
end

```